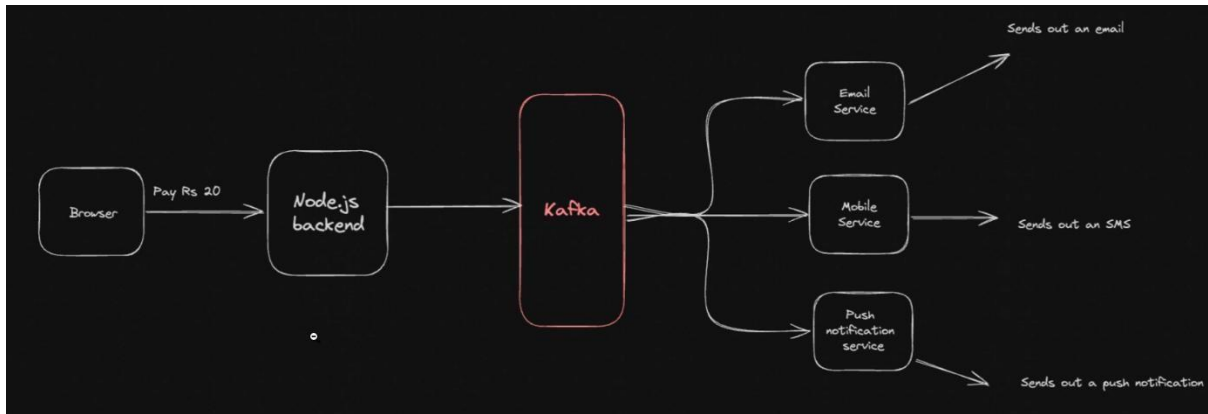


Kafka

Think of **Apache Kafka** as a **high-speed, fault-tolerant messaging backbone** used to move data between systems in real time.



What is Kafka?

Apache Kafka is a system to **send, store, and process streams of events (messages)**.

Key difference from typical messaging:

- It **stores messages (like a log)**
- Consumers can **read anytime, even later**

Kafka vs Redis Pub/Sub

Compare with **Redis Pub/Sub**:

Core Difference

Feature	Kafka	Redis Pub/Sub
Storage	Stored on disk	Not stored
Replay	Yes (read old events)	No
Feature	Kafka	Redis Pub/Sub
Reliability	High	Low

Scaling	Massive	Limited
Use case	Event streaming	Real-time notifications

Simple Truth

- **Redis Pub/Sub = live radio** ◦ Miss it → gone
- **Kafka = Netflix** ◦ Watch anytime (replay)

Where Kafka is Used in Backend

1. Microservices Communication

Instead of services calling each other directly:

Order Service → Kafka → Payment Service
→ Email Service
→ Analytics

Why:

- Loose coupling
- Easy to scale

2. Event-Driven Systems Example:

- user_registered
- order_created

Kafka handles all these events centrally

3. Async Processing (Important for you) Your booking system:

- User books ticket
- Instead of blocking:

- Send event → Kafka ◦

Worker processes later

Why:

- Faster API response
- Better performance under load

4. Logging & Analytics Pipelines

- Track user activity
- Stream logs
- Feed dashboards

5. Data Pipelines

Move data between:

- DB → Kafka → Data warehouse

Why Use Kafka (Real Reason)

Not “because it’s cool”, but when you need:

1. Reliability

Messages are **not lost** it can let you replay messages

2. Replay You can:

- Debug issues
- Reprocess events

Redis can’t do this

3. Scalability

Handles **huge traffic and distributed system** so even if any node goes down **data is present in other active nodes** no data lost and till nodes are up

4. Decoupling

Services don't depend on each other

Event Streaming

Event streaming = continuously sending and processing small pieces of data (events) as they happen, in real time

In super simple terms

- An **event** = something that happens
(user clicked, order placed, payment done)
- **Streaming** = these events keep flowing continuously

So:

Event streaming = a live flow of events

Jargons

1. Cluster (system brain)

Group of Kafka servers working together **Why it matters for processing:**

- Load is shared
 - If one server dies → processing continues
-

2. Broker (worker machine)

One Kafka server that:

- Stores messages

- Serves them to consumers **Processing angle:**
 - Consumers don't talk to "Kafka"
 - They talk to specific brokers to fetch data
-

3. Topic (event stream)

A continuous log of events **Important**

shift:

- It's NOT a queue (remove after read)
- It's a **log (append-only)** **Example:**

ticket_booked -----

user1 booked user2

booked user3

booked New

events keep getting

added

Old events stay (till retention)

4. Partition (parallel processing engine) **CRITICAL**

A topic is split into partitions

Why?

- Each partition = ordered log
- Multiple partitions = parallel processing **Example:**

Topic: ticket_booked

Partition 0 → user1, user4

Partition 1 → user2, user5

Partition 2 → user3, user6

Now:

- 3 consumers can process in parallel
- Each handles one partition

5. Producer (event creator)

Sends events to a topic **Processing**

detail:

- Producer decides **which partition** ○ Round-robin OR ○ Based on key (e.g., userId)

This affects ordering + scaling

6. Consumer (processor, not just reader)

Reads AND processes events

Important:

- Kafka doesn't "push" → consumer **pulls**
- Consumer controls speed **Example:**

Read event → process payment → send email

7. Consumer Group (load balancing system)

Multiple consumers working together **Rule:**

- One partition → one consumer in a group **Example:**

3 partitions, 3 consumers:

C1 → Partition 0

C2 → Partition 1

C3 → Partition 2

This is how scaling works

8. Offset (processing checkpoint) VERY IMPORTANT

Each message has a number

But real meaning:

"How far has this consumer processed?" Example:

offset 5 → last processed offset

6 → next to process **Why**

powerful:

- Crash? → resume from offset 6
- Want reprocess? → reset offset

Kafka doesn't track processing → **consumer does**

9. Retention (data lifecycle)

Kafka keeps data for a fixed time **Processing**

impact:

- You can:
 - Reprocess events
 -

Debug issues ◦ Build new services

later **Example:**

- Retention = 7 days
 - Even processed data is still there
-

Full Processing Flow (Real Backend) Let's

simulate:

Step 1: Event Created Booking

service (producer):

ticket_booked(user1)

Step 2: Stored in Kafka

- Goes to topic ticket_booked
- Stored in Partition 1
- Gets offset = 42

Step 3: Consumer Picks It

Payment service (consumer):

read offset 42

Step 4: Processing Happens

→ process payment

→ update DB

→ maybe emit new event

Step 5: Offset Commit mark

offset 42 as done

If crash before this → reprocess

If after → move forward

Key Insight (this is what most miss) Kafka

does NOT:

- Execute logic
- Know if processing succeeded

It only: **stores + serves events**

Consumers are responsible for:

- Processing
- Reliability
- Idempotency

Final Mental Model

Kafka =

“A durable log of events that consumers process at their own speed, with full control over progress”

Quick Reality Check (important)

If you're thinking:

“Why not just Redis Pub/Sub?” Because:

- No offsets → no recovery
- No retention → no replay
- No partitions → limited scaling [Implementing kafka in node .js](#)

Step 1 — Create Project

```
mkdir kafka-express-ts cd
```

```
kafka-express-ts npm init -y
```

Just initializes Node project

Step 2 — Install Dependencies [npm](#)

```
install express kafkajs
```

```
npm install -D typescript ts-node-dev @types/node @types/express
```

Why:

- express → API
- kafkajs → Kafka client (simple, production-ready)
- ts-node-dev → run TS directly

Step 3 — Setup TypeScript

```
npx tsc --init
```

Update tsconfig.json (important minimal config):

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "CommonJS",
```

```

    "rootDir": "./src",
    "outDir": "./dist",
    "esModuleInterop": true,
    "strict": true
  }}

```

Ste

p 4

—

Pr

oje

ct

Str

uct

ur

e

src

/

└─ index.ts

└─ kafka/

| └─ producer.ts

| └─ consumer.ts

Keep Kafka logic separate (clean backend design)

Step 5 — Start Kafka (KRaft) Use

Docker (simplest):

docker-compose.yml

version: "3.8"

services:

kafka:

image: bitnami/kafka:latest

ports: - "9092:9092"

environment:

- KAFKA_ENABLE_KRAFT=yes
- KAFKA_CFG_NODE_ID=1

- KAFKA_CFG_PROCESS_ROLES=broker,controller
 - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
 - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
 - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
 - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
- Run: docker-compose up -d

What this does:

- Starts Kafka **without ZooKeeper**
- One node acting as:
 - broker (stores data)
 - controller (manages cluster)

Step 6 — Kafka Producer (send events)

```
// src/kafka/producer.ts
import { Kafka } from "kafkajs";

const kafka = new Kafka({
  clientId: "booking-app",
  brokers: ["localhost:9092"],
});

const producer = kafka.producer();

export const connectProducer = async () => {
  await producer.connect();
  console.log("Producer connected");
};

export const sendMessage = async (message: string) => {
  await producer.send({
    topic: "ticket_booked",
    messages: [{ value: message }],
  });
};
```

What this does:

- Connects to Kafka
- Sends event → topic ticket_booked

Step 7 — Kafka Consumer (process events)

```
// src/kafka/consumer.ts import
{ Kafka } from "kafkajs"; const
kafka = new Kafka({  clientId:
"booking-app",
  brokers: ["localhost:9092"],
});

const consumer = kafka.consumer({ groupId: "booking-group" });

export const connectConsumer = async () => {
  await consumer.connect();

  await consumer.subscribe({
    topic: "ticket_booked",
    fromBeginning: true,
  });

  await consumer.run({
    eachMessage: async ({ message }) => {
      const value = message.value?.toString();
      console.log("Processing:", value);

      // simulate processing
      // → DB update
      // → send email
    },
  });
};
```

What this does:

- Subscribes to topic
- Continuously pulls events
- Processes them

Step 8 — Express API

```
// src/index.ts
import express from "express";
import { connectProducer, sendMessage } from "../kafka/producer"; import
{ connectConsumer } from "../kafka/consumer";

const app = express();
app.use(express.json());

app.post("/book", async (req, res) => {
  const { user } = req.body;

  await sendMessage(`ticket booked by ${user}`);

  res.json({ message: "Booking event sent" });
});

const start = async () => {
  await connectProducer();
  await connectConsumer();

  app.listen(3000, () => {
    console.log("Server running on port 3000");
  });
};

start();
```

Flow:

1. API hit /book
2. Producer sends event
3. Consumer receives & processes

Step 9 — Run App

Add script:

```
"scripts": {  
  "dev": "ts-node-dev src/index.ts"  
}
```

Run: `npm`

`run dev`

You'll see:

Processing: ticket booked by prajwal **Group id**

What is Group ID?

Group ID = name of a consumer team `groupId`:

`"booking-group"`

All consumers with same `groupId`:

work **together**

```
import { kafka } from "./kafka";
```

```
const consumer = kafka.consumer({ groupId:  
"order-service", // 💧 THIS IS IMPORTANT  
});
```

```
export const startConsumer = async () => { await  
consumer.connect();
```

```
await consumer.subscribe({  
topic: "test-topic",  
fromBeginning: true,  
});
```

```
await consumer.run({  
eachMessage: async ({ topic, partition, message }) => {  
console.log({ groupId: "order-service",  
partition,
```

```
value: message.value?.toString(),  
});  
}  
}); }
```

Why groupId exists?

Because Kafka doesn't send messages to "all consumers"

It sends messages to **consumer groups**

Load Balancing (actual meaning)

Kafka splits work between consumers in the same group

Example

Topic: ticket_booked

Partitions: 3 Consumers:

C1 (group: booking-group)

C2 (group: booking-group)

C3 (group: booking-group)

What Kafka does:

C1 → Partition 0

C2 → Partition 1

C3 → Partition 2

Each consumer gets **different data**

This is **load balancing**

Important Rule

1 partition = only 1 consumer in a group

If more consumers than partitions

Partitions = 2

Consumers = 3

C1 → P0

C2 → P1

C3 → idle

One consumer does nothing

If different groupIds

C1 (group: email-service)

C2 (group: analytics-service)

BOTH get SAME messages

No load balancing here

This is **fan-out**

Mental Model

- **Same groupId** → split work (load balance)
- **Different groupId** → duplicate work (independent processing)

groupId decides whether consumers share work or get full copies

Subtle but Important Insight Kafka is

not doing “smart processing”

It only:

- Assigns partitions
- Tracks offsets per group

Your app must:

- Handle failures
- Ensure idempotency

What is a Consumer Group?

A consumer group = team of consumers working together groupId:

"booking-group"

All consumers in same group:
share the work

Partitions

A partition = a slice of a topic Instead of
one big stream, Kafka splits it:

ticket_booked topic

Partition 0

Partition 1

Partition 2

Why partitions exist?

Parallel processing + scaling Each
partition:

- Has ordered messages
- Can be processed independently

```
import { kafka } from "../kafka";
```

```
const admin = kafka.admin();
```

```
export const createTopic = async () => {  
  await admin.connect();
```

```
  await admin.createTopics({
```

```
    topics: [
```

```
    {
```

```
      topic: "orders", numPartitions: 3, // 💧 HERE  
      partitions are defined
```

```
}, ],  
});  
await admin.disconnect();
```

Force to one partition **await**

```
producer.send({ topic:  
"orders",  
messages: [  
{  
value: "Order 1", partition: 0, // 💧  
force to partition 0  
},  
{  
value: "Order 2",  
partition: 1,  
}, ],  
});
```

3. Partition Key (MOST IMPORTANT CONCEPT)

A key is a value you attach to a message so Kafka can decide:
which partition this message should go to

This is what you'll use in real systems.

```
export const sendWithKey = async () => {  
await producer.connect();  
  
await producer.send({  
topic: "orders",  
messages: [  
  {  
    key: "user-123", // 💧 SAME KEY = SAME PARTITION  
value: "Order A",  
  },  
  {  

```

```
    key: "user-123",  
    value: "Order B",  
  },  
],  
});  
  
await producer.disconnect();  
};
```

Purpose of Key

1. Keep related data together

Same key → same partition Example:

user1 → Partition 0 user1
→ Partition 0 user1 →
Partition 0

2. Maintain order

Kafka guarantees order only inside a partition So:

user1: book → pay →
confirm

All in correct order because same partition

Without Key

messages: [{ value: "ticket booked" }]

Kafka uses round-robin:

msg1 → P0 msg2
→ P1 msg3 → P2

Problem:

Order is NOT guaranteed

Real Backend Decision (Important)

Booking system

Good key:

key: userId

Why:

- **All events of one user stay together**
- **Order is preserved**

Bad key: key:

"booking"

All messages go to ONE partition

One consumer overloaded

Advanced Insight (very important) Key

affects TWO things:

1. Ordering

2. Load distribution

Bad distribution example key =

country

If:

- **90% users from India**

One partition overloaded

Others idle

How to Choose Key (Real Thinking) Ask:

“What needs ordering?”

- Per user → use userId
- Per order → use orderId
- No ordering → no key

How They Work Together (IMPORTANT)

Rule:

Each partition is assigned to ONLY ONE consumer in a group

Example 1 (Perfect balance)

Partitions = 3

Consumers = 3 (same group)

C1 → Partition 0

C2 → Partition 1

C3 → Partition 2

All consumers busy

Maximum parallel processing

Example 2 (Less consumers)

Partitions = 3

Consumers = 2

C1 → Partition 0 + 1 C2

→ Partition 2

One consumer handles multiple partitions

Example 3 (More consumers)

Partitions = 2

Consumers = 3

C1 → Partition 0

C2 → Partition 1 C3

→ idle

Extra consumers are useless

Why This Matters (Real Backend Thinking)

1. Scaling consumers

Want more throughput?

Adding more consumers blindly won't help

Increase partitions

2. Ordering guarantee

Order is guaranteed ONLY within a partition Example:

Partition 0 → user1 events (ordered)

Partition 1 → user2 events (ordered)

Across partitions → no order guarantee

3. Load balancing

Kafka automatically distributes partitions across consumers

You don't write load balancing logic

Real Example (Your Booking System)

Topic: ticket_booked Partitions:

3

Consumers (same group: booking-workers):

- **Worker 1 → processes some bookings**
- **Worker 2 → processes others**
- **Worker 3 → processes others**

All bookings processed in parallel

Important Insight (Most People Miss)

Kafka doesn't balance messages

It balances partitions

Subtle but Critical

If one partition gets heavy traffic:

Partition 0 → 90% data

Partition 1 → 5%

Partition 2 → 5%

One consumer becomes bottleneck

So partitioning strategy matters (key selection)

Final Mental Model

- **Partition = unit of parallelism**
- **Consumer Group = workers sharing load**
- **Kafka assigns partitions → not messages**

Purpose of Consumer Groups + Partitions

Using Apache Kafka

The purpose is:

To process large amounts of data fast, reliably, and in parallel

Problem 1: One consumer is too slow If

you have:

- **10,000 bookings/sec**
- **1 consumer**

It will choke

Solution: Partitions Split

data:

Partition 0

Partition 1

Partition 2

Now work can be done in parallel

Solution: Consumer Group Kafka

automatically assigns:

C1 → Partition 0

C2 → Partition 1

C3 → Partition 2

No overlap

No duplicate processing

Balanced workload

Problem 3: Avoid duplicate processing If

all consumers read all messages:

Same booking processed multiple times

Solution: Group logic

In same group:

- **Each message processed once**

Problem 4: System should scale easily Traffic

increases → what do you do?

Solution:

- **Add partitions → more parallelism**
- **Add consumers → more workers**

Horizontal scaling

How are partitions decided..?

Short Answer

Partitions are decided in TWO ways:

1. You (developer) decide how many partitions a topic has
2. Kafka decides which partition each message goes to (based on key or round-robin)

1. How many partitions? (YOU decide)

When creating a topic:

--partitions 3

You are saying:

- “I want max 3 parallel consumers”

How to choose number?

Think like this:

Partitions = max parallel processing capacity Example:

- 3 partitions → max 3 consumers working
- 10 partitions → max 10 consumers

Don't guess randomly Base it

on:

1. Throughput (load)

- High traffic → more partitions

2. Consumers

- Want 5 workers → at least 5 partitions

3. Future scaling

- Increasing partitions later is harder

Important Behavior

If topic already exists:

- Kafka will NOT recreate it
- No error (safe to call every startup)

Three caeses

Core Rule (remember this)

1 partition → only 1 consumer (per group) Everything comes from this.

Case 1: Equal Partitions = Equal Consumers Example:

- Partitions = 3
- Consumers = 3

C1 → P0

C2 → P1

C3 → P2

What happens:

- Perfect load balance
- All consumers active
- Maximum parallel processing

Meaning:

This is the ideal setup

Case 2: More Partitions, Less Consumers Example:

- Partitions = 4
- Consumers = 2

C1 → P0 + P1

C2 → P2 + P3

What happens:

- Consumers handle multiple partitions
- Still works fine

But:

- Each consumer has more load
- Parallelism is limited by number of consumers

Meaning:

System is under-utilized You

can scale later:

- Add more consumers → better distribution

Case 3: More Consumers, Less Partitions Example:

- Partitions = 2
- Consumers = 4

C1 → P0

C2 → P1

C3 → idle

C4 → idle

What happens:

- Extra consumers do nothing
- Waste of resources

Meaning:

Partitions limit scaling

You CANNOT scale beyond partitions

Real Insight (this is what matters)

Kafka does NOT distribute messages

It distributes partitions

Quick Comparison

Case	Efficiency	Scaling
Equal = Equal	Best	Fully utilized
More partitions	Good	Can scale later Wasted
More consumers	Bad	

Subtle Insight (interview level) Even if:

Partitions = 4

Consumers = 2

Load may STILL be uneven if:

- **One partition has more traffic**

So:

- **Partition count $\neq$ perfect balance**
-

Final Mental Model

- **Partitions = “lanes”**
- **Consumers = “workers”**

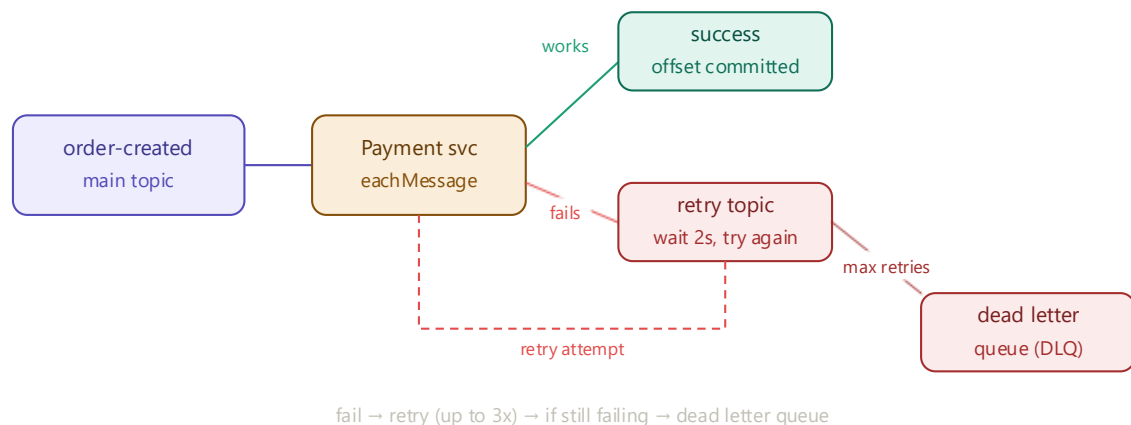
Workers can handle multiple lanes

But one lane cannot have multiple workers

One-Line Summary

Consumers scale only up to the number of partitions

If `processPayment` throws an error — Kafka doesn't know. Your service just crashes or skips the message silently. That order never gets paid. No retry, no alert, nothing. In production that means lost money.



The pattern is: try → fail → wait → try again → fail again → wait longer → try again → if still failing → give up and send to DLQ.

The waiting between retries is called **exponential backoff** — each retry waits longer than the last:

attempt 1 fails → wait 1 second → retry

attempt 2 fails → wait 2 seconds → retry

attempt 3 fails → wait 4 seconds → retry

attempt 4 fails → give up → dead letter queue

Why wait longer each time? Because most failures are temporary — a database is briefly overloaded, a network hiccup, a third party API is slow. Waiting gives the problem time to resolve itself.

Dead Letter Queue (DLQ)

A DLQ is just a normal Kafka topic with a special purpose — it holds messages that failed all retry attempts.

topic: order-created → normal processing

topic: order-created.retry → messages being retried

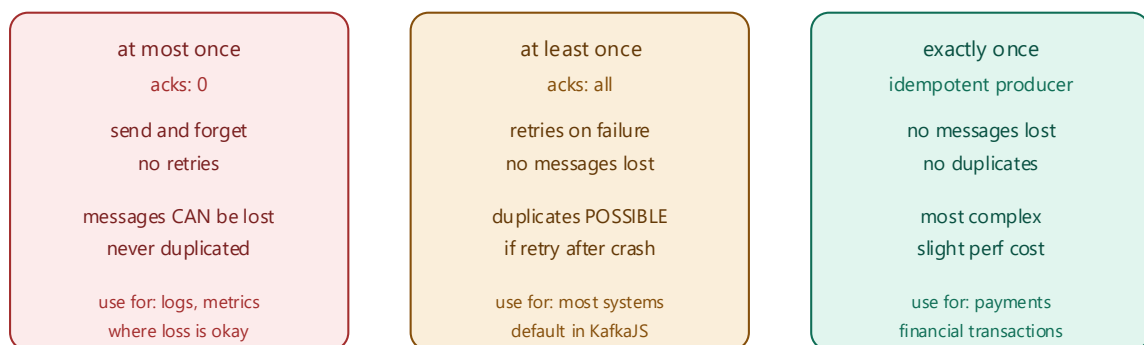
topic: order-created.dlq → messages that failed everything

Why not just log the error and move on? Because if you skip the message, **that order is silently lost**. With a DLQ:

- The message is safely stored
- An engineer can inspect it later — why did it fail?
- You can replay DLQ messages after fixing the bug
- Nothing is ever permanently lost

Exactly-once delivery

Kafka gives you three delivery guarantees to choose from:



we will implement: at least once + idempotency check (industry standard)

The duplicate problem happens like this:

1. Payment service processes order-123 successfully
2. Payment service crashes BEFORE committing the offset
3. Kafka thinks the message was never processed
4. Payment service restarts → processes order-123 AGAIN
5. Customer charged twice ✨

The fix is **idempotency** — making your processing safe to run twice. You check "have I already processed this orderId?" before doing the work. If yes, skip it. This is the industry standard approach.

Graceful shutdown

Right now if you Ctrl+C your service, it dies mid-message. The fix is catching the shutdown signal and letting the current message finish first:

Ctrl+C pressed



finish current message ← don't interrupt



disconnect from Kafka ← clean disconnect



process exits ← safe

Without this, you get partial processing — payment half-done, offset not committed, chaos.